第一境

1.1

1. 绝对路径是从文件系统的根目录 / 开始写起的完整路径。相对路径是从当前工作目录出发，描述如何到达目标位置的路径。
2. cd ../../../var/backups/turing/

1.2

1. argc(argument count)表示命令行参数的个数。它至少为 1，因为程序名本身也算一个参数。
   argv(argument vector)，是一个指向字符串数组的指针。 argv[0]指向程序名（或程序被调用时的路径）;argv[n]指向第n个命令行参数;argv[argc]标准规定为 NULL，表示参数列表结束。

2. 针对命令cd ../../../var/backups/turing/ argc=2（程序名 + 1 个参数）;argv[0]指向字符串 "cd"（程序名）; argv[1]指向字符串 "../../../var/backups/turing/"; argv[2]=NULL（因为只有两个参数，argv[2] 为空）。

第二境

2.1

1. <stdio.h><string.h><stdlib.h><time.h>较常见，<dirent.h><sys/types.h><sys/stat.h>较少见。

2.

<dirent.h> 关键结构体：

DIR：目录流，表示一个已打开的目录。

struct dirent：目录项，包含 d_name（文件名）等字段。

关键函数：

opendir()：打开目录，返回DIR*。

readdir()：读取目录中的下一个条目，返回 struct dirent*。

closedir()：关闭目录流。

作用：用于遍历目录、获取目录中的文件名列表。

<sys/stat.h> 关键结构体：

struct stat：保存文件的元数据，如 st_mode（权限和类型）、st_size（大小）、st_mtime（最后修改时间）等。

关键函数：

stat()：根据路径获取文件状态。

fstat()：根据文件描述符获取文件状态。

lstat()：类似 stat，但不跟随符号链接。

chmod()：修改文件权限。

mkdir()：创建目录。

作用：获取和修改文件的属性信息（大小、权限、时间戳等）。

2.2 void parse_permissions(mode_t mode, char *perms) { char rwx[] = {'r', 'w', 'x'}; // TO DO mode_t mask = 0400; // 从用户读权限位开始（八进制 0400 = 二进制 100 000 000） for (int i = 0; i < 9; i++) { perms[i] = (mode & mask) ? rwx[i % 3] : '-'; mask >>= 1; // 检查下一位 } // END OF TO DO perms[9] = '\0'; // 字符串结尾 }

第三境

3.1

1.

Inode（索引节点）保存的是文件的元数据，主要包括：

文件类型：普通文件、目录、符号链接、字符设备、块设备、FIFO、套接字等。

权限：读、写、执行，分别针对所有者、组、其他。

链接数：有多少个目录项指向这个 inode（硬链接计数）。

所有者：用户 ID（UID）和组 ID（GID）。

文件大小：以字节为单位。

时间戳：

atime(最后访问时间)，

mtime(最后修改时间)，

ctime(最后状态改变时间)。

数据块指针：指向文件内容所在磁盘块的指针，包括直接块、一级间接、二级间接、三级间接等。

其他属性：如文件系统特定的标志、扩展属性等。

2.

node 中不包含文件的信息：

文件名：文件名存储在目录项中，目录项将文件名映射到 inode 号。同一个 inode 可以有多个文件名（硬链接）。

文件路径：路径是目录层次结构的逻辑表示，不是 inode 的一部分。

文件内容：内容存储在数据块中，inode 只包含指向这些数据块的指针。

3.

.代表当前目录自身，..代表父目录。 在根目录 / 下，由于根目录没有父目录，所以/ 下的 . 和 .. 都指向根目录自身。 它们的 inode 编号相同，都等于根目录的 inode 号。在常见的 ext2/3/4 文件系统中，根目录的 inode 号通常是 2（inode 1 保留给坏块）。 因此，在 / 下，. 和 .. 的 inode 编号都是 2。

3.2

#include <stdio.h> #include <stdlib.h> #include <string.h> #include <unistd.h> #include <sys/stat.h> #include <dirent.h>

ino_t get_inode(const char *path) { struct stat statbuf; if (stat(path, &statbuf) == -1) { perror("stat failed"); exit(EXIT_FAILURE); } return statbuf.st_ino; }

int main() { char *names[256]; // 保存各级目录名（从当前目录到根目录，逆序） int count = 0;

```
while (1) {
    ino_t cur_inode = get_inode(".");
    ino_t parent_inode = get_inode("..");

    // 如果当前目录的 inode 与父目录相同，说明已到达根目录
    if (cur_inode == parent_inode) {
        break;
    }

    // 保存当前目录的 inode，用于在父目录中查找它的名字
    ino_t target_inode = cur_inode;

    // 切换到父目录
    if (chdir("..") == -1) {
        perror("chdir failed");
        exit(EXIT_FAILURE);
    }

    // 打开父目录，遍历查找与 target_inode 匹配的目录项
    DIR *dir = opendir(".");
    if (!dir) {
        perror("opendir failed");
        exit(EXIT_FAILURE);
    }

    struct dirent *entry;
    char *name = NULL;
    while ((entry = readdir(dir)) != NULL) {
        if (entry->d_ino == target_inode) {
            name = strdup(entry->d_name);
            break;
        }
    }
    closedir(dir);

    if (name == NULL) {
```

```
            fprintf(stderr, "Error: could not find directory name for inode %lu\n",
                    (unsigned long)target_inode);
            exit(EXIT_FAILURE);
        }

        names[count++] = name;
    }

    // 输出绝对路径
    if (count == 0) {
        printf("/\n");
    } else {
        for (int i = count - 1; i >= 0; i--) {
            printf("/%s", names[i]);
        }
        printf("\n");
    }

    // 释放动态分配的内存
    for (int i = 0; i < count; i++) {
        free(names[i]);
    }

    return 0;
```

}

最终境

#include <stdio.h> #include <string.h> #include <dirent.h> #include <sys/stat.h> #include <unistd.h>

// 递归查找指定文件，若找到则打印其相对路径 void find_file(const char *current_path, const char *target_name) { DIR *dir; struct dirent *entry; struct stat statbuf;

```
    if (!(dir = opendir(current_path))) {
        return; // 无法打开目录则跳过
    }

    while ((entry = readdir(dir)) != NULL) {
        // 忽略当前目录 . 和父目录 .. 防止无限死循环
        if (strcmp(entry->d_name, ".") == 0 || strcmp(entry->d_name, "..") == 0) {
            continue;
        }

        char full_path[1024];
        snprintf(full_path, sizeof(full_path), "%s/%s", current_path, entry->d_name);

        if (stat(full_path, &statbuf) == -1) continue;
```

```
        // TO DO

        // 检查当前 entry 的文件名是否与 target_name 匹配
        if (strcmp(entry->d_name, target_name) == 0) {
            printf("%s\n", full_path);
        }

        // 如果当前 entry 是一个目录，则递归查找
        if ((statbuf.st_mode & S_IFMT) == S_IFDIR) {
            find_file(full_path, target_name);
        }

        // END OF TO DO
    }
    closedir(dir);
```

}